

Real-time GS Mapping

Anonymous CVPR submission

Paper ID 9619

Abstract

001 *3D Gaussian Splatting (3DGS) has become a prominent*
002 *representation for real-time novel view rendering, finding*
003 *widespread use across various visual applications. How-*
004 *ever, due to its structural complexity and dense optimiza-*
005 *tion requirements, building a high-quality 3DGS map is*
006 *predominantly an offline process. This presents a critical*
007 *barrier for robotics and wearable augmented-reality sys-*
008 *tems, which strictly demand online mapping to construct*
009 *and refine environments on the fly. Furthermore, because*
010 *"real-time" performance is an inherently ambiguous met-*
011 *ric that fluctuates depending on specific hardware capa-*
012 *bilities, simply targeting an absolute processing speed is*
013 *an insufficient goal. Instead, we study fast photometric*
014 *and geometric convergence in 3DGS SLAM: the rate at*
015 *which the appearance and structure of newly observed re-*
016 *gions improve under a sequential input stream and a given,*
017 *bounded mapping budget. Two bottlenecks prevent this on-*
018 *line convergence: tracking-oriented view selection dilutes*
019 *available updates, and photometric losses exploit incorrect*
020 *opacity in observed free space. To address these, we ex-*
021 *pand the mapping-view set based on completed mapping*
022 *updates and introduce Entropy-Regularized Count Balanc-*
023 *ing (ERCB) for equitable appearance refinement. For ge-*
024 *ometric convergence, a causal free-space Carve loss sup-*
025 *presses spurious opacity in front of observed surfaces. Re-*
026 *gardless of the given computational environment, our for-*
027 *mulation forces each unit of online optimization to produce*
028 *reliable appearance and geometry immediately after obser-*
029 *vation.*

030 1. Introduction

031 3D Gaussian Splatting (3DGS) has established itself as a
032 highly effective representation for photorealistic novel view
033 synthesis [12]. By modeling scenes with explicit, colored
034 Gaussian primitives and rendering them via a tile-based
035 differentiable rasterizer, it provides a compelling combina-
036 tion of high visual fidelity and real-time rendering speeds.
037 Consequently, 3DGS is increasingly utilized across a di-

verse range of visual applications. To achieve this level of
quality, the representation requires careful optimization of
numerous parameters—including position, scale, rotation,
color, and opacity—for millions of primitives. Since this
optimization process is computationally intensive and re-
lies on dense multi-view constraints, 3DGS frameworks are
predominantly trained offline using completely collected
datasets.

However, autonomous robots and wearable augmented-
reality systems (e.g., Aria glasses) are fundamentally bound
to the present moment. For these active agents, high recon-
struction quality achieved only after an exhaustive offline
refinement stage is inadequate; they intrinsically require on-
line mapping to construct and reliably refine a map as new
observations continuously arrive. Adapting 3DGS to this
online constraint exposes a critical vulnerability. While re-
cent SLAM systems strive to process streams quickly, "real-
time" operation is an inherently ambiguous metric that fluc-
tuates heavily depending on the available GPU hardware.
Therefore, rather than engineering a system merely to meet
an arbitrary processing speed, the true objective shifts to
achieving *fast photometric and geometric convergence* for
practical field deployments. Regardless of the specific hard-
ware environment, the system is forced to make a newly ob-
served region structurally and visually reliable as quickly as
possible under whatever computational budget is given.

Building a reliable online Gaussian map inherently re-
quires two complementary objectives: establishing a geo-
metrically sound starting point and rapidly optimizing it un-
der strict computational limits. Recent advancements have
approached these goals from various angles. For instance,
EDGS [13] demonstrates that dense image correspondences
can provide a robust geometric initialization, laying the
groundwork for high-quality reconstruction. At the op-
timizer level, 3DGS-LM introduces second-order solvers
to accelerate convergence [7], while systems like CaRtGS
and PGSR incorporate allocation strategies and geometric
consistencies to maximize the efficiency of available com-
pute [2, 5]. While these approaches successfully highlight
the necessity of learning efficiency and structural reliability,
relying solely on advanced optimizers or dense initialization

079 is insufficient. To truly pursue fast convergence in a stream-
080 ing environment, the fundamental question shifts from *how*
081 to optimize to *when* and *which* observations should enter the
082 training pipeline.

083 The rate of map convergence relies critically on the
084 view selection policy. Current online systems, such as
085 Photo-SLAM [9] and VIGS-SLAM [30], heavily depend on
086 frontend-selected keyframes for map optimization. While
087 bounding computation to local keyframe windows (as seen
088 in MonoGS and HI-SLAM2 [16, 28]) efficiently reuses
089 selected observations, discarding the intermediate frames
090 leaves valuable texture and viewpoint information unex-
091 ploited. Conversely, naively injecting all available frames
092 into the optimization pool dilutes the limited update bud-
093 get, leaving each view insufficiently refined. Uniform sam-
094 pling from an ever-growing pool fails to compensate for the
095 varying entry times of new observations, systematically de-
096 laying the convergence of recently added regions. Hence,
097 achieving rapid photometric improvement requires decou-
098 pling the mapping-view set from the tracking trajectory. By
099 dynamically coupling the inclusion of new observations to
100 actual completed optimization work and meticulously bal-
101 ancing training opportunities across views, a limited com-
102 putational budget can be strictly directed to bring the ap-
103 pearance of newly observed regions to high quality sooner.

104 A strong geometric starting point is essential, but it
105 does not guarantee that structures remain accurate during
106 subsequent color-driven optimization. Due to the shape-
107 radiance ambiguity of alpha compositing, misplaced Gaus-
108 sians can effortlessly replicate target colors by forming
109 semi-transparent clouds in empty space [3, 24, 25]. One
110 might assume that geometric losses can resolve this by sim-
111 ply translating (i.e., updating the positions of) these float-
112 ing artifacts back to the correct surface. However, relying
113 on positional gradients exposes a fundamental flaw in the
114 3DGS architecture. Because the rasterizer depends on strict
115 depth-sorting [12], shifting a Gaussian along the camera ray
116 frequently alters its sorting order relative to its neighbors,
117 causing abrupt, discontinuous spikes in the transmittance
118 integral. Furthermore, it is inherently ambiguous whether
119 a floater should be pushed toward the foreground object or
120 pulled into the background. Attempting to spatially nav-
121 igate this highly non-convex landscape wastes critical on-
122 line updates. Therefore, rather than struggling to move mis-
123 placed Gaussians, an online method should directly control
124 their opacity, explicitly suppressing density in observed free
125 space to rapidly eliminate spurious structures without rely-
126 ing on unstable positional gradients [6, 26, 27].

127 To achieve this fast convergence within a bounded on-
128 line budget, we build upon a DROID-SLAM-based fron-
129 tend [22]. While recent works have explored dense geo-
130 metric initialization [13], efficient second-order solvers [7],
131 and compute-aware optimization allocations [2, 5], trans-

lating these foundations into continuous map improvement
requires strict control over which observations enter train-
ing. Therefore, we tightly couple our mapping policy to the
actual optimization progress. For photometric refinement,
we introduce an Optimization-Guided View Growth strat-
egy that expands the mapping-view set strictly at the rate of
completed updates. This is complemented by our Entropy-
Regularized Count Balancing (ERCB) sampler, which dy-
namically corrects severe training deficits among views
while preserving the diversity of sampling without replace-
ment. For geometric refinement, we apply a causal free-
space Carve loss that directly acts on opacity. By contin-
uously penalizing opacity where depth evidence dictates
empty space, we enforce structural consistency immedi-
ately after a region is observed.

We validate whether this unified architecture fundamen-
tally accelerates map convergence rather than merely offer-
ing a fast rendering speed. Through controlled incremental
evaluations, we compare our rendering quality and struc-
tural accuracy against recent state-of-the-art online systems,
notably VIGS-SLAM [30]. To strictly isolate the efficacy of
our proposed mapping policies from the inherent benefits
of our dense initialization, we keep the initialization proto-
col completely frozen across all ablation studies. This rig-
orous experimental design explicitly demonstrates how our
view scheduling and opacity-directed constraints extract su-
perior, highly converged map quality from the exact same
computational budget. Our contributions are summarized
as follows:

- Building upon a dense, frontend-driven initialization, we
formulate an online 3DGS mapping framework optimized
strictly for *fast photometric and geometric convergence*
under limited compute.
- We propose an Optimization-Guided View Growth pol-
icy that paces the inclusion of causal frames according to
completed mapping work.
- We introduce Entropy-Regularized Count Balancing
(ERCB) to correct severe optimization imbalances while
preserving the essential diversity of sampling without re-
placement.
- We design a causal, depth-supported Carve loss that tar-
gets opacity directly, rapidly suppressing free-space ar-
tifacts and bypassing the severe gradient discontinuities
associated with positional optimization.

2. Related Work

2.1. Efficient Convergence under Limited Opti- mization Budgets

The cost of radiance-field reconstruction can be reduced
either by making each update cheaper or by reducing the
number of updates needed to obtain a useful representation.
Explicit grids and compact encodings accelerated neural-

field optimization, while 3DGS made full-resolution differentiable rendering practical through an explicit primitive representation [12]. Depth supervision can further reduce ambiguity and training time when only a small set of observations is available [3]. Within 3DGS, Taming 3DGS controls primitive growth and improves the optimization kernels under explicit resource limits [15]. Photo-SLAM uses geometry-based densification and Gaussian-pyramid learning to improve photorealistic mapping under incremental optimization [9]. Recent geometry-aware online mapping revisits scale initialization and densification because offline 3DGS heuristics are brittle under short online budgets [14].

These works establish that initialization, representation capacity, and the cost of an optimization step strongly affect attainable quality. Our focus is complementary: we study the joint *trajectory* of photometric and geometric quality after observations arrive. We therefore compare methods not only at a fixed endpoint, but also at matched elapsed time and matched completed updates. This operational definition is important in a growing map, for which there is no single static objective whose asymptotic convergence fully describes online behavior.

2.2. Online 3DGS SLAM and Observation Allocation

Neural dense SLAM systems introduced continuous optimization of a scene representation during camera tracking. iMAP selects rays using information gain [21], NICE-SLAM improves scalability with hierarchical features [29], and Co-SLAM combines coordinate and sparse parametric encodings with global ray sampling [23]. DROID-SLAM instead estimates pose and dense depth through recurrent updates and differentiable dense bundle adjustment [22]; this frontend is particularly useful for initializing Gaussian maps from dense geometric estimates.

3DGS-SLAM systems differ in how tracking, geometry, and appearance mapping are coupled. Gaussian Splatting SLAM and SplatTAM track against Gaussian maps and select local or overlapping keyframes for mapping [11, 16]. Photo-SLAM decouples conventional localization from progressive photorealistic Gaussian mapping [9], RTG-SLAM restricts updates to unstable primitives for scalable reconstruction [18], and Splat-SLAM jointly refines poses, depth, and Gaussians [19]. VIGS-SLAM combines a DROID-style recurrent visual-inertial frontend with a Gaussian mapping backend [30]. These systems primarily use keyframes and local visibility to determine which observations participate in mapping, reflecting the close historical coupling between tracking and map optimization.

The closest observation-allocation work is CaRtGS, which identifies uneven keyframe re-optimization in a growing keyframe pool and assigns additional iterations us-

ing per-keyframe loss [5]. Active-view methods such as FisherRF instead choose which camera observation should be acquired from expected information gain [10]. Our setting assumes a fixed causal stream: the question is which already observed images should be used by the mapper, and how a fixed number of updates should be distributed among them. We expand the mapping-view set beyond tracking keyframes according to completed mapping work and use update counts as an inexpensive coverage signal. ERCB does not estimate information gain or guarantee non-convex 3DGS convergence; it is a bounded correction for severe optimization imbalance in a growing view set. Its use of sampling without replacement is motivated by random-reshuffling methods [17, 20], while the changing online objective prevents directly inheriting their fixed-sum convergence guarantees.

2.3. Geometric Convergence and Floater Suppression

Photometric reconstruction is geometrically under-constrained because many opacity and depth configurations can explain similar colors. DS-NeRF treats sparse depth as a distributional constraint along each ray [3], and Mip-NeRF 360 regularizes the distribution of rendering weights with a distortion loss [1]. For Gaussian reconstruction, 2DGS uses surface-oriented disks together with depth-distortion and normal-consistency terms [8], while PGSR introduces planar depth and multi-view geometric consistency [2]. These methods improve geometric accuracy, but are chiefly evaluated in offline reconstruction where long optimization schedules and global access to posed images are available.

Floater-removal methods intervene in primitive selection or in the coupling between geometry and appearance. TrimGS prunes Gaussians using multi-view contribution and geometric criteria [4]. EFA-GS emphasizes low-frequency structure to reduce floating artifacts caused by poor initialization [24]. StableGS attributes persistent floaters to weak opacity gradients under color supervision and separates geometry and appearance opacity paths so that stronger depth constraints do not erase visual detail [25]. Online systems also use scale, opacity, and visibility constraints: RTG-SLAM distinguishes stable opaque primitives from low-opacity ones [18], and CaRtGS combines adaptive optimization with opacity regularization [5].

Our geometry objective is designed for the shorter horizon and restricted compute of online mapping. Rather than globally relocating or pruning every suspect primitive, the current Carve instantiation uses causal depth to ask a one-sided question: is a Gaussian’s opacity supported in space known to be free? Contradicted opacity is suppressed, while unobserved space and possible surfaces behind the first hit are left unconstrained. Because this operation can improve

free-space precision by deleting structure, we evaluate it jointly with surface accuracy, completeness, and held-out appearance. This distinguishes fast geometric convergence from aggressive opacity removal.

3. Method

Our goal is to improve the rendering and geometric quality of the Gaussian map under the limited mapping updates available in online 3DGS SLAM. Instead of letting keyframe selection fix both the set of training views and which of them each mapping iteration sees, we decouple the two and drive each from optimization progress already observed—the number of completed mapping updates and how many times each view has been selected. We first add the frames between keyframes to the training views and let the number of completed mapping updates, rather than the keyframe rule, determine how fast that set grows (§3.1). We then sample views from a distribution that favors under-selected views while retaining stochasticity, and use sampling without replacement to reduce repetitive view selections across consecutive mapping updates (§3.2). Finally, we constrain free-space violations that photometric supervision does not directly constrain, using only observations that have already arrived, so that geometry converges sooner (§3.3 - Detailed sentences need to be modified).

3.1. Optimization-Guided View Growth

Offline 3DGS optimizes a fixed set of views, whereas GS-SLAM must update the map as new observations arrive while sharing computation with tracking. Mapping can therefore perform only a limited amount of optimization during an online run. Several real-time GS-SLAM systems optimize the map using a bounded set of keyframes [15,16,17]. However, keyframe selection is not designed solely to provide dense photometric supervision for map optimization [cite]. Our results in Table X show that including frames between keyframes improves map quality under the same number of mapping updates. At the same time, removing earlier views can reduce supervision for regions that are weakly covered by the remaining set and makes it harder to distribute optimization across the full trajectory. We therefore retain previously added training views and allow the training set to grow as new observations become available.

Growing the training set introduces a competing effect. Adding views provides more supervision, but the same mapping updates must then be distributed over a larger set, reducing how often each view can be revisited. The appropriate rate at which views should be added therefore depends on how much optimization can actually be performed. In an online setting, this amount cannot be determined exactly in advance: when a new frame arrives, both the remaining length of the sequence and the number

of mapping updates that will eventually be completed are unknown. The decision to include that frame must nevertheless be made online. We therefore make the growth of the training set follow the optimization work that has actually been completed, rather than a schedule fixed before the run.

Let $S(t)$ denote the number of mapping updates completed by time t , counting an update only after it has finished. Let $\mathcal{A}_t$ be the training-view set at time t , and let $N_0 = |\mathcal{A}_0|$ be the number of initial views. We use a single parameter, κ , to specify the number of completed mapping updates required for each additional view. The number of views added by time t is then constrained by

$$\kappa (|\mathcal{A}_t| - N_0) \leq S(t). \quad (1)$$

Solving Eq. (1) for the training-set size gives

$$N_{\max}(t) = \min \left(M(t), N_0 + \left\lfloor \frac{S(t)}{\kappa} \right\rfloor \right), \quad (2)$$

where $M(t)$ is the number of views available by time t . The minimum prevents the training set from exceeding the views that have already arrived, and the floor accounts for views being added one at a time. When additional views are available, one view can therefore be added for every κ completed mapping updates. Measured in completed mapping updates, this growth rate is determined only by κ : it does not decrease as the training set becomes larger and does not depend on how views already in the set are selected for optimization. This separates the decision of when to add new views from the decision of which existing views to optimize; Section 3.2 addresses the latter. ““

3.2. Entropy-Regularized View Sampling

The training-view set grows over time as new observations become available and mapping optimization proceeds. A simple strategy is to sample uniformly from the views currently in the set. Although this gives every available view the same probability at a given time, it does not account for when each view entered the training set. Views added early remain eligible for selection for longer and can therefore receive substantially more mapping updates, whereas views added later have fewer opportunities to be selected. If this difference accumulates, optimization can become unevenly distributed along the trajectory, leaving less frequently optimized regions with lower reconstruction quality. We therefore want to give more opportunities to views that have received fewer updates. A purely deterministic least-count rule, however, removes the randomization commonly used in stochastic optimization to reduce systematic ordering effects [cite]. This is particularly relevant for SLAM, where nearby camera frames often contain highly similar observations. Our goal is therefore to favor under-selected views while retaining randomness in the selection process.

Let s index mapping-view selections, and let N_s be the number of views available for training at selection step s . Let n_i denote the number of times view i has been selected for a mapping update. We denote the mean selection count by

$$\bar{n} = \frac{1}{N_s} \sum_i n_i.$$

Because the scale of the raw count variance changes as selections accumulate, we measure the dispersion relative to the current mean count:

$$\Phi(\mathbf{n}) = \frac{1}{2N_s\bar{n}} \sum_{i=1}^{N_s} (n_i - \bar{n})^2. \quad (3)$$

A smaller Φ indicates that the accumulated selection counts are more evenly distributed across the current training views. Let $T = \sum_i n_i = N_s\bar{n}$ be the total number of selections made so far. For a single selection step, with the training set held fixed, selecting view i once more changes the dispersion by

$$\Delta_i\Phi = \Phi(\mathbf{n} + \mathbf{e}_i) - \Phi(\mathbf{n}) = \frac{n_i}{T+1} + C_s, \quad (4)$$

where C_s is independent of the selected view. Thus, views with smaller selection counts produce a smaller increase in Φ . Greedily minimizing this change would recover a deterministic least-count rule. Instead, we use $\Delta_i\Phi$ to define a probability distribution over the available views.

Let p_i denote the probability of selecting view i for the next mapping update. We seek a distribution that favors views with smaller $\Delta_i\Phi$ without concentrating all probability on a single view. We express these two objectives jointly as

$$\min_{\mathbf{p} \in \Delta} \sum_i p_i \Delta_i\Phi - \tau H(\mathbf{p}), \quad (5)$$

where $H(\mathbf{p}) = -\sum_i p_i \log p_i$, Δ denotes the probability simplex, and $\tau > 0$ controls the strength of the entropy term. The first term favors views that have been selected less often, while the second discourages the distribution from becoming overly concentrated. Solving Eq. (5) under the constraint $\sum_i p_i = 1$ gives

$$p_s(i) = \frac{\exp(-\Delta_i\Phi/\tau)}{\sum_j \exp(-\Delta_j\Phi/\tau)}. \quad (6)$$

Substituting Eq. (4) removes the term shared by all views and yields

$$p_s(i) = \frac{\exp(-\beta_s n_i)}{\sum_j \exp(-\beta_s n_j)}, \quad \beta_s = \frac{1}{\tau(T+1)}. \quad (7)$$

Views with fewer previous selections therefore receive higher probability, while every available view retains a

nonzero probability of being selected. For a fixed τ , β_s decreases as the total number of selections grows, so the influence of a fixed count difference is gradually reduced relative to the accumulated selection history.

Equation (7) defines the distribution for a single view selection. If each mapping update samples independently with replacement, however, a small number of high-probability views can be selected repeatedly over a short interval while other views receive no update. To reduce such short-term repetition, we select K distinct views without replacement for the next K mapping updates. At the beginning of each group, we compute the sampling probabilities from the current selection counts and use them to select K distinct views. Each selected view is then used once, and the updated counts are used to compute the distribution for the next group. This preserves the preference for under-selected views while ensuring that the same view is not repeatedly selected within the group. The parameter K therefore determines the interval over which repeated selections are prevented: $K = 1$ reduces to single-view sampling, whereas larger K increases the variety of views used over consecutive mapping updates. We evaluate the effect of K in Sec. 4.

3.3. Causal Ray-Space Geometry Supervision

Photometric supervision constrains the composited color but does not uniquely determine where opacity lies along a ray: a premature Gaussian can reproduce the observed color while occupying known free space. A conventional rendered depth loss is also insufficient because it first compresses all contributors to one mean depth, allowing separated front and back modes to cancel. We therefore supervise the first-termination process induced by Gaussian compositing using only depth observations available at the current time. This gives two complementary constraints: *Carve* suppresses termination before the observed surface, while *Hit* places termination mass at that surface.

Ray termination. For a depth-ordered set of Gaussian contributors to ray r , let α_i^0 be the base opacity, G_{ri}^{2D} the projected Gaussian value at the pixel, and z_{ri} its camera-space depth. The effective alpha, incoming transmittance, and termination weight are

$$a_{ri} = \alpha_i^0 G_{ri}^{2D}, \quad T_{ri} = \prod_{j < i} (1 - a_{rj}), \quad w_{ri} = T_{ri} a_{ri}. \quad (8)$$

The remaining no-hit mass is $T_{\text{end},r} = \prod_i (1 - a_{ri})$, and $\sum_i w_{ri} + T_{\text{end},r} = 1$. We equivalently introduce a categorical first-termination index I_r with $p(I_r = i | \Theta) = w_{ri}$ and $p(I_r = \emptyset | \Theta) = T_{\text{end},r}$. This is an interpretation of deterministic alpha-compositing mass, not a stochastic renderer.

For each multi-view-verified keyframe ray, VIGS provides a BA-refined depth D_r . Its latent first-surface depth Z_r^* is represented by an external posterior

$$q_r(z) = p(Z_r^* = z \mid \mathcal{O}_r) = \mathcal{N}(z; D_r, \tau_r^2),$$

$$F_r(b) = \Phi\left(\frac{b - D_r}{\tau_r}\right). \quad (9)$$

where $\mathcal{O}_r$ denotes the front-end observations. We stop gradients through D_r and τ_r , keeping this evidence independent of the Gaussian map parameters Θ .

Free-space likelihood (Carve). Given a surface at z and a safety margin m_r , the target event is

$$\mathcal{E}_r^F(z) = \{\text{no model termination before } z - m_r\}. \quad (10)$$

Its conditional likelihood is exactly the transmittance $p(\mathcal{E}_r^F(z) \mid Z_r^* = z, \Theta) = T_\Theta(r, z - m_r)$. Taking the posterior expectation of its conditional NLL gives

$$\mathcal{L}_{\text{C-ENLL}} = \frac{1}{|\mathcal{R}_D|} \sum_{r \in \mathcal{R}_D} \sum_i \gamma_{ri} [-\log(1 - a_{ri})], \quad (11)$$

$$\gamma_{ri} = p(Z_r^* > z_{ri} + m_r \mid \mathcal{O}_r) = 1 - F_r(z_{ri} + m_r),$$

where $\mathcal{R}_D$ is the set of verified depth rays. Thus, a Gaussian is penalized in proportion to the posterior probability that it lies in observed free space. With geometry detached, minimizing Eq. (11) monotonically decreases only its opacity.

Alternatively, marginalizing the compatibility event before taking the NLL gives

$$P_{\text{C-MARG},r} = \int q_r(z) T_\Theta(r, z - m_r) dz$$

$$= T_{\text{end},r} + \sum_i w_{ri} F_r(z_{ri} + m_r), \quad (12)$$

$$\mathcal{L}_{\text{C-MARG}} = -\frac{1}{|\mathcal{R}_D|} \sum_{r \in \mathcal{R}_D} \log(P_{\text{C-MARG},r} + \epsilon).$$

Equation (12) is the exact NLL of the stated compatibility event under an external q_r , whereas Eq. (11) is its posterior-expected conditional NLL. Jensen’s inequality gives $\mathcal{L}_{\text{C-MARG}} \leq \mathcal{L}_{\text{C-ENLL}}$, so the latter is more conservative. Both objectives permit a no-hit solution and therefore remove free-space violations without, by themselves, creating surface coverage.

First-hit likelihood (Hit). Carve specifies where termination should not occur; Hit additionally explains the measured surface. Conditioned on termination at Gaussian i , we use the forward observation model

$$k_{ri} = p(D_r \mid I_r = i, \Theta) = \mathcal{N}(D_r; z_{ri}, \sigma_r^2), \quad (13)$$

where σ_r is calibrated measurement noise, not Gaussian thickness. Marginalizing the latent termination index yields

$$p(D_r, I_r \neq \emptyset \mid \Theta) = \sum_i p(D_r \mid I_r = i, \Theta) p(I_r = i \mid \Theta)$$

$$= \sum_i w_{ri} k_{ri}, \quad (14)$$

$$\mathcal{L}_{\text{HIT}} = -\frac{1}{|\mathcal{R}_D|} \sum_{r \in \mathcal{R}_D} \log\left(\sum_i w_{ri} k_{ri} + \epsilon\right).$$

Writing $A_r = \sum_i w_{ri}$ decomposes each ray loss into $-\log A_r - \log \sum_i (w_{ri}/A_r) k_{ri}$: the first term demands a finite hit, and the second aligns the complete termination distribution with D_r . Consequently, near-surface components gain termination mass, premature occluders lose opacity when they block a better downstream explanation, and components with nonzero mixture responsibility receive position gradients toward D_r . Unlike Carve, the canonical Hit objective therefore optimizes opacity, position, and projected shape.

Non-overlapping Carve-Hit objective. Adding global Carve and Hit losses independently can count the same front transmittance twice because every w_{ri} already contains T_{ri} . For an exact factorization, define a free-space boundary $B_r = D_r - m_r$, an upper surface boundary U_r , and

$$\mathcal{F}_r = \{i : z_{ri} < B_r\}, \quad \mathcal{S}_r = \{i : B_r \leq z_{ri} \leq U_r\}. \quad (15)$$

Let $T_r^F = \prod_{i \in \mathcal{F}_r} (1 - a_{ri})$ and reset transmittance inside the surface set,

$$\tilde{w}_{ri}^S = a_{ri} \prod_{\substack{j \in \mathcal{S}_r \\ z_{rj} < z_{ri}}} (1 - a_{rj}). \quad (16)$$

Since $w_{ri} = T_r^F \tilde{w}_{ri}^S$ for $i \in \mathcal{S}_r$, the restricted first-hit NLL factorizes as

$$\mathcal{L}_{\text{C+H}}(r) = -\log T_r^F - \log\left(\sum_{i \in \mathcal{S}_r} \tilde{w}_{ri}^S k_{ri} + \epsilon\right). \quad (17)$$

The first term suppresses opacity only in front of the surface boundary; the second creates and aligns termination mass only in the local surface region. Resetting transmittance at B_r prevents double penalization. Replacing the hard first term with Eq. (11) gives an uncertainty-aware composite objective, but no longer the exact factorization in Eq. (17).

Online integration. We evaluate these objectives only on spatially balanced, multi-view-verified depth rays. The existing rasterizer already computes $(z_{ri}, G_{ri}^{2D}, a_{ri}, T_{ri}, w_{ri})$

in its front-to-back contributor loop; we accumulate the required likelihood in the same pass and combine it with the mapping loss before a single backward pass. The additional work is therefore linear in the number of selected ray–Gaussian pairs and does not require a second full-frame render. Carve stops gradients through depth, ordering, and footprint and updates opacity only, avoiding unreliable position updates across sorting discontinuities. Hit retains full gradients for surface contributors. The factorized objective applies these routes separately to $\mathcal{F}_r$ and $\mathcal{S}_r$.

With the unchanged VIGS mapping loss $\mathcal{L}_{\text{base}}$, the three candidate objectives are

$$\begin{aligned}\mathcal{L}_{\text{map}}^{\text{C}} &= \mathcal{L}_{\text{base}} + \lambda_{\text{C}} \mathcal{L}_{\text{C}}, \\ \mathcal{L}_{\text{map}}^{\text{H}} &= \mathcal{L}_{\text{base}} + \lambda_{\text{H}} \mathcal{L}_{\text{HIT}}, \\ \mathcal{L}_{\text{map}}^{\text{C+H}} &= \mathcal{L}_{\text{base}} + \lambda_{\text{C+H}} \mathcal{L}_{\text{C+H}},\end{aligned}\quad (18)$$

where $\mathcal{L}_{\text{C}}$ denotes either $\mathcal{L}_{\text{C-ENLL}}$ or $\mathcal{L}_{\text{C-MARG}}$ as selected by the controlled ablation.

4. Experiments

4.1. Setup

Datasets.

DATASETS 9 lines
[not yet chosen — see notes/datasets/CURRENT.md] One self-captured Aria sequence plus one or two public sets, each with a clause saying what makes it different (the corpus does this: cartgs, MonoGS). State the held-out split exactly, in the manner of SparseGS’s “every eighth image as testing view” — ours are the frames in `test.txt`, which no method uses for supervision. If a sequence is excluded, say why in place (MonoGS: “Since Replica is designed for RGB-D ... we hence use it for RGB-D evaluation only”).

Metrics.

METRICS 10 lines
Quality: PSNR, SSIM, LPIPS on held-out views. Resources: wall-clock time, number of streaming updates, number of admitted views, and peak and final model size — peak matters because a method that oversamples and prunes is otherwise indistinguishable from one that stays within budget (Taming3DGS’s Peak #G). Define ρ_{H} here, as cartgs defines IPF in its setup: the entropy of the draw normalised by that of an unweighted draw, so 1.0 means no randomness was given up. Name the tools (evo, torchmetrics) the way cartgs does, with footnote URLs. All numbers are mean $\pm$ std over three seeds. Numbers reported under a post-processed-pose protocol are not directly comparable to ours.

Implementation details.

Table 1. Quantitative rendering comparison on RPNG, UTM, and Aria. Values are averaged over the valid scenes of each dataset. Best results are shown in bold.

Metric	Method	RPNG	UTMM	Aria
PSNR $\uparrow$	VIGS-SLAM	22.49	18.73	22.99
	Ours	24.09	19.25	25.44
SSIM $\uparrow$	VIGS-SLAM	0.7460	0.6441	0.7840
	Ours	0.8052	0.6634	0.8331
LPIPS $\downarrow$	VIGS-SLAM	0.2426	0.4216	0.4555
	Ours	0.1789	0.4123	0.3451

IMPLEMENTATION DETAILS

9 lines

Hardware in one clause (a single RTX 5090). Retain the VIGS-SLAM defaults and list only what changed — κ , β , K — which is how cartgs writes it; these three values appear nowhere else in the paper. Then the contract, in one sentence: each method receives the sensor stream in real time and performs zero optimizer updates after the last frame, so the compute budget is the duration of the stream itself; poses and depths are the online-estimated ones, never the post-processed trajectory. One sentence that the same configuration is used on every sequence — nothing retuned per scene, which is what makes Table 2 worth reading (exp59: a configuration tuned on one trajectory lost 1.85 dB on another). Results with a post-stream refinement tail are in the supplement.

Baselines.

BASELINES — CRITERIA, NOT A LIST

7 lines

Photo-SLAM, MonoGS, CaRtGS and VIGS-SLAM. Give the criteria that select them, as MonoGS does: they run in real time from monocular input and build a photorealistic map; code or published numbers exist; and they do not presuppose a depth sensor or a post-processed pose — which is what excludes the RGB-D-only systems. State the provenance rule here and mark it in the tables: reproduced by us, cited from the paper, or failed.

4.2. Main Results

P1 — RESULTS POINTER

7 lines

Point at Table ?? and Table 2 at once, state the highlighting convention, and state the provenance marks here (reproduced by us / cited / failed) — cartgs opens its results with exactly this sentence, and CoMapGS marks reproduced numbers with a dagger.

Table 2. [TABLE 2] One configuration, every sequence. Nothing was retuned per scene (§4.1). Mean $\pm$ std over three seeds.

Sequence	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$
[sequences TBD]	—	—	—

574

P2 — SAME STREAM, SAME TIME

18 lines

Read the quality columns of Table ?? *against* the resource columns, not apart from them: the claim is not that we render better but that we render better *from the same stream in the same time*. Note what the peak-versus-final gap says about methods that oversample before pruning. **Close with what we did not use** — MonoGS’s “without requiring any deep priors” move: no post-processed trajectory, no depth sensor, and no optimization after the stream ends. This is where the contract declared in §4.1 earns its keep; declaring it and never cashing it in would be wasted.

575

P3 — WHERE THE SCENES DIVERGE

14 lines

Table 2, one configuration across every sequence. Say where the self-captured Aria sequence and the public one behave differently **and why** — CoMapGS does this per dataset rather than reporting an average and moving on. [scenes not yet chosen; see notes/datasets/CURRENT.md]

576

P4 — WHERE WE LOSE

11 lines

Name the losses before a reviewer does. CoMapGS: “second only to ReconFusion in the 3-view case. This difference is likely due to ...”; chen2026cover concedes that a random baseline is already strong. Where a method beats us, say so and say what it uses that we contractually do not — a depth sensor, a post-processed pose, or time after the stream. [fill once numbers exist; the slot is reserved so the concession is not an afterthought]

Table 3. Ablation on the source of mapping supervision. Both policies use 60 mapping updates per keyframe. Results are averaged over 19 scenes. Best results are shown in bold.

Metric	Policy	RPNG (8)	UTMM (7)	Aria (4)	All (19)
PSNR $\uparrow$	Keyframes only	23.41	20.25	28.50	23.31
	+ in-between frames	23.06	21.12	29.25	23.65
SSIM $\uparrow$	Keyframes only	—	—	—	—
	+ in-between frames	—	—	—	—
LPIPS $\downarrow$	Keyframes only	—	—	—	—
	+ in-between frames	—	—	—	—

Table 4. Ablation on view ordering under different mapping budgets. The budget denotes the number of mapping updates per interval. Results are averaged over 19 scenes. Best results are shown in bold.

Metric	Policy	15 upd./interval	30 upd./interval	60 upd./interval
PSNR $\uparrow$	Random reshuffling	20.77	22.22	22.88
	ERCB	21.39	22.33	23.11
SSIM $\uparrow$	Random reshuffling	—	—	—
	ERCB	—	—	—
LPIPS $\downarrow$	Random reshuffling	—	—	—
	ERCB	—	—	—

4.3. Ablations

577

OPENER — scope, data, seal, pointer (3 sentences)

5 lines

What varies and on what data — one sentence, as SparseGS opens (“We ablate our method on the Mip-NeRF360 dataset 12-view setting”). **Then the seal:** every configuration receives the same stream, the same real-time budget, the same seeds, and no per-scene retuning; only the policy under test differs (Taming3DGS seals the Gaussian count, TIDI-GS seals budget *and* camera paths — we have three confounds to seal, so we name all three). **Then point at Table 3, Table 4 and Fig. 1 at once.**

578

View growth.

579

ITEM 1 — Imrs shape: purpose+\$ref, table, rows in order, one number

11 lines

Restate the purpose with the section reference in the sentence, as Imrs writes it (“We propose a view sampling approach in Sec. 4.3 to ensure that...”). §3.1 lets completed service, not the keyframe rule, set how fast the pool may grow. Then read the two replacement rows of Table 3 in order. **Report the κ sweep in prose, not in the table** — hyperparameter sweeps sit outside the ablation table in the corpus (Taming3DGS puts its budget sweep in Fig. 1) — and **state the $\kappa=16$ anomaly here rather than omitting it [22.163 dB, −5.545 against trend; full sweep in the supplement]**. Close on one number.

580

Count balancing.

581

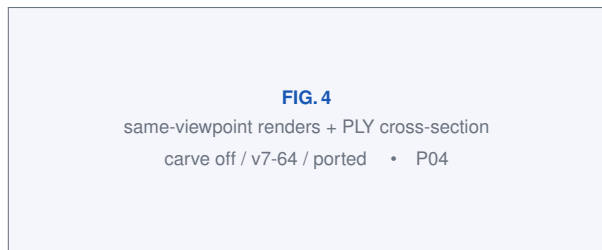


Figure 1. Carving removes free-space Gaussians.

ITEM 2 — REGIME FRAMING first, matches §3.2 15 lines
 Concede the regime first, as chen2026cover does (“only as good as the random baseline in very sparse initialization regimes”): where blocks are long, random reshuffling is already strong, and the gain appears where $K \ll N_t$. Read Table 4 down K before across β , so the reader sees which of the two axes is the larger effect — lmr prints the table in which its own contribution loses to batch size and says so. Then the proxy-versus-goal divergence from §3.2, with both numbers. [the $K=128, \beta=0$ arm does not exist yet, so the effect of β alone is unmeasured; say this in the text — an empty cell alone will read as an oversight] Close on one number.

Carving.

ITEM 3 — same shape, matches §3.3 9 lines
 [teammate] Purpose with the §3.3 reference, the removal row of Table 3, then Fig. 1 for what the number does not show — SparseGS pairs its floater-pruning row with a qualitative figure for exactly this reason. Close on one number.

CLOSING — read the table as a whole 3 lines
 Removing any one of the three costs quality, and the three losses differ in kind — which is the claim of §3’s opening recovered as a measurement, not restated.

5. Conclusion

In this paper, we introduce the closed-form n^{th} moment of conics and prove that the centroid of a distorted ellipse is expressed by a linear combination of the original moments. Based on these results, we developed an unbiased estimator of circular patterns under distortion for camera calibration. Using this estimator, circular pattern-based calibration overcomes its algorithmic limitation and outperforms checkerboard-based calibration.

Acknowledgments. This work was supported by the NRF grant funded by MSIP (2020R1C1C1006620) and IITP grant funded by MSIT (No.2022-0-00480).

References

- [1] Jonathan T. Barron, Ben Mildenhall, Dor Verbin, Pratul P. Srinivasan, and Peter Hedman. Mip-NeRF 360: Unbounded anti-aliased neural radiance fields. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 5470–5479, 2022. 3
- [2] Danpeng Chen, Hai Li, Weicai Ye, Yifan Wang, Weijian Xie, Shangjin Zhai, Nan Wang, Haomin Liu, Hujun Bao, and Guofeng Zhang. PGSR: Planar-based gaussian splatting for efficient and high-fidelity surface reconstruction. In *European Conference on Computer Vision*, 2024. 1, 2, 3
- [3] Kangle Deng, Andrew Liu, Jun-Yan Zhu, and Deva Ramanan. Depth-supervised NeRF: Fewer views and faster training for free. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022. 2, 3
- [4] Lue Fan, Yuxue Yang, Minxing Li, Hongsheng Li, and Zhaoxiang Zhang. Trim 3D gaussian splatting for accurate geometry representation, 2024. arXiv:2406.07499. 3
- [5] Dapeng Feng, Zhiqiang Chen, Yizhen Yin, Shipeng Zhong, Yuhua Qi, and Hongbo Chen. CaRtGS: Computational alignment for real-time gaussian splatting slam. *IEEE Robotics and Automation Letters*, 2025. 1, 2, 3
- [6] Antoine Guédon and Vincent Lepetit. SuGaR: Surface-aligned gaussian splatting for efficient 3d mesh reconstruction and high-quality mesh rendering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 7154–7164, 2024. 2
- [7] Lukas Höllein, Aljaž Božič, Michael Zollhöfer, and Matthias Nießner. 3DGS-LM: Faster gaussian-splatting optimization with Levenberg–Marquardt. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 26740–26750, 2025. 1, 2
- [8] Binbin Huang, Zehao Yu, Anpei Chen, Andreas Geiger, and Shenghua Gao. 2D gaussian splatting for geometrically accurate radiance fields. *ACM Transactions on Graphics*, 2024. 3
- [9] Huajian Huang, Longwei Li, Hui Cheng, and Sai-Kit Yeung. Photo-SLAM: Real-time simultaneous localization and photorealistic mapping for monocular, stereo, and RGB-D cameras. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 21584–21593, 2024. 2, 3
- [10] Wen Jiang, Boshu Lei, and Kostas Daniilidis. FisherRF: Active view selection and mapping with radiance fields using fisher information. In *European Conference on Computer Vision*, 2024. 3
- [11] Nikhil Keetha, Jay Karhade, Krishna Murthy Jatavallabhula, Gengshan Yang, Sebastian Scherer, Deva Ramanan, and Jonathon Luiten. SplatAM: Splat, track & map 3D gaussians for dense RGB-D slam. In *Proceedings of the IEEE/CVF*

- Conference on Computer Vision and Pattern Recognition, pages 21357–21366, 2024. 3
- [12] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3D Gaussian Splatting for Real-Time Radiance Field Rendering. *ACM Transactions on Graphics*, 42(4):1–14, 2023. 1, 2, 3
- [13] Dmytro Kotovenko, Olga Grebenkova, and Björn Ommer. EDGS: Eliminating densification for efficient convergence of 3DGS. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 41065–41076, 2026. 1, 2
- [14] Thai Luu, Quan Tran, Hieu Phan, and Tuan Dang. Geometry-aware online mapping for 3D gaussian splatting slam, 2026. arXiv:2608.14902. 3
- [15] Saswat Subhajyoti Mallick, Rahul Goel, Bernhard Kerbl, Francisco Vicente Carrasco, Markus Steinberger, and Fernando De La Torre. Taming 3DGS: High-quality radiance fields with limited resources. In *European Conference on Computer Vision*, 2024. 3
- [16] Hidenobu Matsuki, Riku Murai, Paul H. J. Kelly, and Andrew J. Davison. Gaussian splatting slam. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 18039–18048, 2024. 2, 3
- [17] Konstantin Mishchenko, Ahmed Khaled, and Peter Richtárik. Random reshuffling: Simple analysis with vast improvements. In *NeurIPS*, 2020. 3
- [18] Zhexi Peng, Tianjia Shao, Yong Liu, Jingke Zhou, Yin Yang, Jingdong Wang, and Kun Zhou. RTG-SLAM: Real-time 3D reconstruction at scale using gaussian splatting. In *ACM SIGGRAPH 2024 Conference Papers*, 2024. 3
- [19] Erik Sandström, Keisuke Tateno, Michael Oechsle, Michael Niemeyer, Luc Van Gool, Martin R. Oswald, and Federico Tombari. Splat-SLAM: Globally optimized RGB-only slam with 3D gaussians, 2024. arXiv:2405.16544. 3
- [20] Ohad Shamir. Without-replacement sampling for stochastic gradient methods. In *NeurIPS*, 2016. 3
- [21] Edgar Sucar, Shikun Liu, Joseph Ortiz, and Andrew J. Davison. imap: Implicit mapping and positioning in real-time. In *ICCV*, 2021. 3
- [22] Zachary Teed and Jia Deng. Droid-slam: Deep visual slam for monocular, stereo, and rgb-d cameras. In *NeurIPS*, 2021. 2, 3
- [23] Hengyi Wang, Jingwen Wang, and Lourdes Agapito. Co-slam: Joint coordinate and sparse parametric encodings for neural real-time slam. In *CVPR*, 2023. 3
- [24] Jianchao Wang, Peng Zhou, Cen Li, Rong Quan, and Jie Qin. Low-frequency first: Eliminating floating artifacts in 3D gaussian splatting, 2025. arXiv:2508.02493. 2, 3
- [25] Luchao Wang, Qian Ren, Kaiming He, Hua Wang, Zhi Chen, and Yaohua Tang. StableGS: A floater-free framework for 3D gaussian splatting, 2025. arXiv:2503.18458. 2, 3
- [26] Zehao Yu, Torsten Sattler, and Andreas Geiger. Gaussian opacity fields: Efficient adaptive surface reconstruction in unbounded scenes. *ACM Transactions on Graphics (ToG)*, 43(6):1–13, 2024. 2
- [27] Zehao Yu et al. PAGaS: Pixel-aligned 1dof gaussian splatting, 2026. 2
- [28] Wei Zhang, Qing Cheng, David Skuddis, Niclas Zeller, Daniel Cremers, and Norbert Haala. HI-SLAM2: Geometry-aware gaussian slam for fast monocular scene reconstruction. *IEEE Transactions on Robotics*, 2025. 2
- [29] Zihan Zhu, Songyou Peng, Viktor Larsson, Weiwei Xu, Hujun Bao, Zhaopeng Cui, Martin R. Oswald, and Marc Pollefeys. Nice-slam: Neural implicit scalable encoding for slam. In *CVPR*, 2022. 3
- [30] Zihan Zhu, Wei Zhang, Moyang Li, Norbert Haala, Marc Pollefeys, and Daniel Barath. VIGS-SLAM: Visual inertial gaussian splatting slam, 2025. arXiv:2512.02293. 2, 3

Write Title Here

718

Supplementary Material

719